

DOCUMENTATION FOR THE RATPOINTS PROGRAM

MICHAEL STOLL

1 Introduction

This paper describes the `ratpoints` program. This program tries to find all rational points within a given height bound on a hyperelliptic curve in the most efficient way possible.

2 History and Acknowledgments

This program goes back to an implementation of the ‘quadratic sieving’ idea by **Noam Elkies** that was around in the early 1990s. My own first contribution was to replace the char arrays that were used to store the sieving information by bit arrays, in 1995. **Colin Stahlke** then made use of the `gmp` library, so that points could be checked exactly, and implemented the selection of sieving primes according to their likely success rate, in 1998. After that, I successively put in numerous improvements (and some bug fixes). For details of how the program works, see Section 8 below.

Along with **Noam Elkies** and **Colin Stahlke**, I would like to thank **John Cremona** and **Sophie Labour** for bug reports and suggestions for improvements. Further thanks are due to **Bill Allombert** for a first implementation of the use of 256-bit (and possibly 512-bit) registers.

3 Availability

The `ratpoints-2.2.3` package can be downloaded from my homepage, see [rat], or from github at <https://github.com/MichaelStollBayreuth/ratpoints>.

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but **without any warranty**; without even the implied warranty of **merchantability** or **fitness for a particular purpose**. See the GNU General Public License for more details.

You should receive a copy of the GNU General Public License along with this program.

If not, see <http://www.gnu.org/licenses/>.

4 Installation

This section describes the installation procedure under Linux.

4.1 Extract the archive

```
> tar xzf ratpoints-2.2.3.tar.gz
```

This sets up a directory `ratpoints-2.2.3` containing the various files that belong to the installation.

Date: May 15, 2023.

4.2 Build the program and library

Do

```
> cd ratpoints-2.2.3
> make all
```

This will build the library `libratpoints.a`, the executable `ratpoints`, and also run `pdflatex` to generate this documentation.

Two of the constants that govern the sieve depend on the machine rather than on the curve. Reasonable values are compiled in, but `make tune` will measure better ones for your machine if you want it to; see Section 7. It is a separate step because it takes several minutes.

By default, the program will use primes up to 127. If you intend to do computations with large height bounds or with curves that you expect to have many points, it may make sense to increase the range of primes. This can be achieved via

```
> make all PRIME_SIZE=8
```

perhaps after a

```
> make distclean
```

to remove the files that were generated previously. The `PRIME_SIZE` argument can be given any value from 5 to 10; otherwise it is taken to be 7. The precise meaning is that the program will work with primes $< 2^s$, when `PRIME_SIZE = s`.

The sieve works on several numerators at a time, in a register whose width is selected by the line

```
CCFLAGS1 = ${CCFLAGS256}
```

in `Makefile`. It can be set to `${CCFLAGS64}` (word-size registers, which works on essentially any machine), `${CCFLAGS128}`, `${CCFLAGS256}` (the default, using 256-bit AVX registers) or `${CCFLAGS512}`.

The `Makefile` notices that the setting has changed and rebuilds everything that depends on it, so a `make distclean` in between is not necessary. Wider registers help the first phase of the sieve, where most of the time goes on a curve with few rational points, but they help the second phase much less and the sieve tables not at all, so the gain is nowhere near proportional to the width: on the author's machine, 64, 128 and 256 bits take 21.3, 15.2 and 12.2 units of time on such a curve.

The 128-bit setting uses the macro `USE_AVX128`. In spite of its name it needs only SSE2 instructions, so it runs wherever the older `USE_SSE` variant did, and it is never slower, because its test for an empty register compares the whole register against zero instead of extracting both halves into general registers. That is worth about 30% of the second phase of the sieve, which comes to 9% of a run long enough for the sieve to dominate it and about 3% of `make test1`. The older variant is still available as `${CCFLAGS128s}`.

You may have to change `-mavx2` in the line

```
CCFLAGS256 = -DUSE_AVX -mavx2
```

into `-mavx` or leave it out altogether, depending on the capabilities of your CPU. You will notice that AVX2 (or AVX) is not supported, when the executable throws an 'unknown instruction' error or similar.

There is also a variant using 512-bit registers, activated by

```
CCFLAGS1 = ${CCFLAGS512}
```

which requires a CPU with AVX512F capability. Note that this variant has so far only been tested indirectly (see the change log entry below), since I have no such CPU available; you should therefore run `make test` before relying on it. It is also worth timing the 256-bit and the 512-bit version against each other on your machine: the sieving loop is limited by memory bandwidth rather than by arithmetic, so wider registers do not automatically pay off, and some Intel server processors reduce their clock frequency when 512-bit instructions are used. The one data point available so far is encouraging, though: on an AMD Ryzen 9 9950X3D (Zen 5 architecture), **Drew Sutherland** measured a speed-up of 13 to 14% over the 256-bit version, for a curve with many rational points.

If you leave out `-mavx512f` and compile with just `-DUSE_AVX512`, then `gcc` will express the 64-byte vectors through narrower operations, so that the code runs on any machine. This is of no use in production (it is slower than the 256-bit version), but it does allow the 512-bit code path to be tested without the corresponding hardware. `gcc` will warn that passing a 512-bit vector argument without AVX512F enabled changes the ABI; this is harmless, as the whole library is built with the same flags.

4.3 Run a test

Run

```
> make test1
```

in the working directory. This will build an executable `rptest` and then run (and time) it. Finally, the output (which was written to a file `rptest.out`) is compared against `testbase`, which contains the output of a sample run. The two should be identical; otherwise the message `Test failed!` will be displayed (on its own on a line).

```
> make test1many
```

does the same with a second collection of curves, in `testdata-many.h`, whose reference output is `testbase-many`. These curves have (comparatively) many rational points, which makes them sieve quite differently from the random curves in `testdata.h`: on a random curve about one numerator in 10^4 survives the first sieving stage, on these up to a hundred times more, so the best division of labour between the two stages is a different one. The two tests take about the same time and between them cover the two regimes that occur in practice; both should be used whenever the parameters are retuned, since a setting that suits one of them can be a poor choice for the other. That is what `make tune` does with them; see Section 7.

```
> make test2
```

calls `ratpoints` on a curve with lots of rational points and with a fairly large height bound, times it, and compares the output to what is expected. This may take a few minutes, so be patient!

```
> make timing
```

times `ratpoints` on some curve with height bound 400 000. This maybe useful for comparisons.

Finally,

```
> make test
```

does all three of the above.

4.4 Install

If you like, you can install the library, executable and header file on your system.

```
> sudo make install
```

The executable is copied to `/usr/local/bin/`, the library to `/usr/local/lib/`, and the header file to `/usr/local/include/`. You can change the `/usr/local` prefix via

```
> make install INSTALL_DIR=...
```

4.5 Debugging

In case you found a bug and would like to find out where it comes from, or if you just want to see exactly what the program is doing, you can do

```
> make debug
```

in the working directory. This will build an executable `ratpoints-debug`, which, when run, will dump loads of output on the screen, so it is best to send the output to a file, which you can then study at leisure.

4.6 Cleaning up

In order to get rid of the temporary files, do

```
> make clean
```

To remove everything except the files from the archive, use

```
> make distclean
```

4.7 Requirements

You need a C compiler (like `gcc`, which is the default specified for the `CC` variable in `Makefile`; if necessary, it can be changed there).

In addition to the standard libraries, the program also requires the `gmp` (GNU multi-precision) library [[gmp](#)].

4.8 List of files

The archive contains the following files.

- `Makefile`
- `ratpoints.h` — the header file for programs using `ratpoints`
- `rp-private.h`, `primes.h` — header files used internally
- `gen_find_points.h.c`, `gen_init_sieve.h.c` — short programs that write additional header files depending on the system configuration
- `sift.c`, `init.c`, `sturm.c`, `find_points.c` — the source code for the `ratpoints` library
- `main.c`, `rpctest.c` — the source code for the `ratpoints` and `rpctest` executables, respectively
- `testdata.h`, `testdata-many.h`, `testbase`, `testbase-many`, `testbase2` — data for the test runs
- `ratpoints-doc-2.2.pdf` — this documentation file
- `gpl-2.0.txt` — the GNU license that applies to this program.

5 How to use ratpoints

5.1 Basic operation

Let

$$C : y^2 = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

be your curve, and let H be the bound for the denominator and absolute value of the numerator of the x -coordinate of the points you want to find. The command to search for the points is then

```
> ratpoints 'a_0 a_1 ... a_{n-1} a_n' H
```

The first argument to `ratpoints` is the list of the coefficients, which have to be integers, are separated by spaces, and are listed starting with the constant term. There is no bound on the size of the coefficients; they are read as multi-precision integers.

The degree n of the polynomial is limited to `RATPOINTS.MAX_DEGREE` (defined in `ratpoints.h`), which is set to 100 by default.

The program will give an error message when the polynomial (considered as a binary form of the smallest even degree $\geq n$) is not squarefree (e.g., if you specify two leading zero coefficients).

The following subsections discuss how to modify the standard behavior. This is achieved by adding options after the two required arguments. Some of the options have arguments, others don't; the ordering of the options and option-argument pairs is arbitrary (except for options with opposite effect, where the last one given counts, and for the specification of the search intervals, which must be in order).

5.2 Early abort

By default, `ratpoints` will search the whole region that you specify and print all the points it finds. If you just want to find *one* point (for example when dealing with 2-covering curves of elliptic curves), you can tell the program to quit after it has found one point via the `-1` option, e.g.,

```
> ratpoints '-18 116 48 -12 30' 60000000 -1
```

If you don't want to see points at infinity, specify the `-i` option. The two options can be combined: `-i -1` will stop the program after it has found one finite point.

5.3 Changing the output

By default, `ratpoints` prints some general information before and after the points, which are given in the form $(x : y : z)$, one per line. Here, $(x : y : z)$ are the coordinates of the point considered as a point in the $(1, \lceil n/2 \rceil, 1)$ -weighted projective plane, which is the natural ambient space for the curve. The coordinates are integers with x and z coprime and $z > 0$, or $z = 0$ and $x = 1$ (for points at infinity).

There are various ways to change this behavior.

- To suppress all output except the points, use `-q` (for *quiet*).
- To add some more output explaining what the program is doing, use `-v` (for *verbose*). This has no effect if the `-q` option is also given.
- To suppress printing of the points, use `-z`.
- If you only want to list the x -coordinates of point pairs rather than individual points, use `-y`.

- There are four options that influence the format in which the points are printed:

`-f format -fs string-before -fm string-between -fe string-after`

The arguments to all of them are strings; `"\n"`, `"\t"`, `"\""` and `"\""` are recognized and do what you expect. The *format* string can contain markers `%x`, `%y`, `%z` that will be replaced with the *x*, *y*, and *z*-coordinate of the point, respectively. The defaults are empty for *string-before*, *string-between* and *string-after*, and `"(%x : %y : %z)\n"` for *format* when `-y` is not specified; otherwise `"(%x : %z)\n"`.

The effect is as follows. Before any point is printed, *string-before* is output. Then every point is printed according to *format*. Between any two points, *string-between* is output, and after the last point, *string-after* is output.

As an example, consider the effect of

`-f "[%x,%y,%z]" -fs "{" -fm "," -fe "}"\n"`

5.4 Restricting the search domain

There are two ways to restrict the domain of the search. The first is to restrict the range of denominators considered. This is done via

`-dl dmin -du dmax`.

For example, to only look for integral points, you can say `-du 1`. By default, the lower limit is 1 and the upper limit is *H*.

The other way is to restrict the search to a union of (closed) intervals. If you only want to search for points in

$$[l_1, u_1] \cup [l_2, u_2] \cup \dots \cup [l_k, u_k],$$

you can specify this in the form

`-l l1 -u u1 -l l2 -u u2 ... -l lk -u uk`.

The first `-l` option is optional; *l*₁ defaults to $-\infty$. Similarly the last `-u` option is optional, and *u*_{*k*} defaults to $+\infty$. The number *k* of intervals is bounded by `RATPOINTS_MAX_DEGREE` (usually, 100).

5.5 Setting the parameters for the sieve

It is possible to change the number of primes that are used in the various stages of the algorithm. This is done by the following options.

`-p M -N N -n n`

This sets the number of (odd) primes that are considered for the sieve to *M*, the number of primes that are actually used for the sieve to *N*, and the number of primes used in the first sieving stage to *n*. The program will, if necessary, reduce *M*, *N*, and *n* (in this order) to ensure that $n \leq N \leq M \leq \text{RATPOINTS_NUM_PRIMES}$. The latter is usually 30 (the number of odd primes $< 2^7$), but will be 53 (the number of odd primes $< 2^8$) if `PRIME_SIZE` is set to 8, etc.

M has a price in memory, because the sieving tables for a prime *p* take *p* rows of *p* bits and so grow with *p*²: the primes below 128 need about 5 MB, those below 1024 about 1.7 GB (twice that with 512-bit registers). Only the primes that *M* actually asks for are reserved, so raising `PRIME_SIZE` in order to have larger primes available costs nothing as long as *M* is left alone.

If `-n` or `-N` is left out, the corresponding number is chosen from the curve, as described in Section 7.1. The two constants that govern that choice can themselves be set:

`-r x -R e`

sets the target number of numerators surviving the first sieving stage per 64-bit word to *x* (a real number; this is what decides *n*), and the number of primes added to *n* to obtain *N* to *e*. The mnemonic is that `-r` and `-R` are to the automatic choice what `-n` and `-N` are to the explicit

one. Both are machine-dependent, and giving them on the command line means they can be retuned without recompiling; the compiled-in defaults are `RATPOINTS_SURVIVORS_PER_WORD` and `RATPOINTS_SP2_EXTRA` in `ratpoints.h`.

There are two more parameters that can be set.

`-F D`

sets the maximal number of ‘forbidden divisors’. If the degree is even and the leading coefficient is not a square, then the denominator of any x -coordinate of a rational point cannot be divisible by a prime p such that the leading coefficient is a non-square mod p . If the leading coefficient is divisible by p , but not a p -adic square, the denominator cannot be divisible by some power of p . The program constructs a list of such forbidden divisors against which to check the denominators, and this option specifies how many of these should be used.

The other option is

`-S S` or `-s .`

In the first form, it sets the number of refinement (interval halving) steps in the isolation of the real components to S . This part of the program computes a Sturm sequence for the polynomial and uses it in order to find a union of intervals that contains the intervals of positivity of the polynomial. This is done by a successive subdivision of the interval $]-\infty, +\infty[$. The number S sets the recursion depth. S can be omitted; then it is given a default value. The option `-s` skips the Sturm sequence computation completely. This also has the effect of removing most of the check for squarefreeness.

5.6 Switching off optimizations

The options `-k` and `-j` can be used to prevent the program from reversing the polynomial, which it usually does if this will lead to faster operation (`-k`), and to prevent the program from using the Jacobi symbol test on the denominators (`-j`). This last test extends the ‘forbidden divisors’ method described in Subsection 5.5 above by computing the Jacobi symbol (l/d) , where l is the leading coefficient and d is the odd and coprime-to- l part of the denominator. If the symbol is -1 , the denominator need not be considered. The use of these options may be questionable, unless you want to see how much performance is gained by using these optimizations.

5.7 Switching off exact testing of points

The option `-x` will prevent the program from checking the potential points that survive the sieve whether they really give rise to rational points. This implies that the points that are output may not actually be points. It also effectively sets the `-y` option, since the y -coordinates are not computed.

5.8 Overriding previous options

The options `-I`, `-Y`, `-Z`, `-S`, `-K`, `-J`, `-X` can be used to cancel the effect of the corresponding lower-case option (and thereby restore the default behavior), when this occurs earlier in the list of options. This may be useful if you want to set a default behavior that is different from what the program does out-of-the-box (e.g., in a shell script), but want the caller to be able to override this change.

6 How to use the library

It is possible to use the `ratpoints` machinery from within your own programs.

The library `libratpoints.a` provides the following functions.


```

long find_points(ratpoints_args*,
                 int proc(long, long, const mpz_t, void*, int*),
                 void*);

void find_points_init(ratpoints_args*);

long find_points_work(ratpoints_args*,
                     int proc(long, long, const mpz_t, void*, int*),
                     void*);

void find_points_clear(ratpoints_args*);

```

The passing of arguments to these functions is via the `ratpoints_args` structure, which is defined as follows.

```

typedef struct { mpz_t *cof; long degree; long height;
                 ratpoints_interval *domain; long num_inter;
                 long b_low; long b_high; long sp1; long sp2;
                 double survivors_per_word; long sp2_extra;
                 long array_size;
                 long sturm; long num_primes; long max_forbidden;
                 unsigned int flags; ...}
    ratpoints_args;

```

The dots at the end stand for additional fields that are used internally and are not of interest here. When calling `find_points`, the `cof` field must point to an array of size `degree + 1` of properly initialized gmp integers; these are the coefficients of the polynomial. The program can alter the values of the integers in this array. To prevent this, set the `RATPOINTS_NO_REVERSE` bit in `flags`; this may result in a loss of performance, though.

`height` gives the height bound. It is an error for the degree or the height bound to be nonpositive; in this case the function returns the value `RATPOINTS_BAD_ARGS`.

The field `domain` must contain a pointer to an array of `ratpoints_interval` structures of length at least `num_inter` plus `degree`. This array gives (in its first `num_inter` entries) the intervals for the search region. The type `ratpoints_interval` is just

```
typedef struct {double low; double up;} ratpoints_interval;
```

the meaning of this should be clear. In `ratpoints`, this is set via the `-l` and `-u` options. Usually, you do not want to restrict the range of *x*-coordinates; then you set `num_inter = 0` (but you still have to fill `domain` with a valid pointer to at least `degree` intervals, unless you set `sturm` to a negative value!) The program may alter the values in the array *domain* points to, unless `sturm` has a negative value (which may lead to a performance loss).

`b_low` and `b_high` carry the lower and upper bounds for the denominator; if non-positive, they are set to 1 and the height bound, respectively. In `ratpoints`, these fields are set by the `-d1` and `-du` options.

`sp1` and `sp2` specify the number of primes to be used in the first sieving stage and in both sieving stages together, respectively. If negative, they are chosen from the curve, as described in Section 7.1; this is what the command line program does when `-n` and `-N` are absent. In `ratpoints`, these fields are set by the `-n` and `-N` options. The fields `survivors_per_word` and `sp2_extra` hold the two constants that govern that choice (options `-r` and `-R`); if negative, the values compiled into `ratpoints.h` are used. Similar statements are true for `num_primes` (option `-p`), `sturm` (options `-s`,

-S) and `max_forbidden` (option -F). The field `array_size` specifies the maximal size (in bit arrays; a bit array contains 32, 64, 128, 256, ... bits, depending on the configuration) of the array that is used in the first sieving stage. If non-positive, it is set to a default value. The various default values are defined at the beginning of the header file `ratpoints.h`, where also the maximal degree of the polynomial is set.

The `flags` field holds a number of bit flags.

- `RATPOINTS_NO_CHECK` — when set, do not check whether the surviving x -coordinates give rise to rational points (set by the `-x` option to `ratpoints`).
- `RATPOINTS_NO_Y` — only list x -coordinates (in the form $(x : z) \in \mathbb{P}^1(\mathbb{Q})$) instead of actual points (with a y -coordinate); this is set by the `-y` option to `ratpoints`.
- `RATPOINTS_NO_REVERSE` — when set, do not allow reversal of the polynomial (set by the `-k` option to `ratpoints`).
- `RATPOINTS_NO_JACOBI` — when set, prevent the use of the Jacobi symbol test (set by the `-j` option to `ratpoints`).
- `RATPOINTS_VERBOSE` — when set, causes the procedure to print some output on what it is doing (set by the `-v` option to `ratpoints`).

There are some other flags that are used internally. One of them might be of interest:

- `RATPOINTS_REVERSED` — when set after the function call, this indicates that the polynomial has been reversed (and the contents of the `coef` array have been modified).

The main vehicle for passing information back to the caller is the `proc` function argument together with the pointer `info`. This function

```
int proc(long x, long z, const mpz_t y, void *info, int *quit)
```

is called whenever a point was found. x , y and z are the coordinates of the point (where y is a gmp integer). `info` is the pointer that was passed to `find_points`; this can be used to store information that should persist between calls to `proc`. If `*quit` is set to a non-zero value, this indicates that `find_points` should abort the point search and return immediately; otherwise the search continues. This is how the `-1` option works. The return value is taken as a weight for counting the points; usually it will be 1.

The usual framework for using `find_points` is as follows.

```
(...)
#include "ratpoints.h"

(...)

mpz_t c[RATPOINTS_MAX_DEGREE+1]; /* The coefficients of f */
ratpoints_interval domain[2*RATPOINTS_MAX_DEGREE];
/* This contains the intervals representing the
   search region */

/*****
 * function that processes the points */
*****/

typedef struct {...} data;

int process(long x, long z, const mpz_t y, void *info0, int *quit)
```

```

{ data *info = (data *)info0;

    (...)

    return(1);
}

/*****
* main
*****/

int main(int argc, char *argv[])
{
    long total, n;
    ratpoints_args args;

    long degree          = 6;
    long height          = 16383;
    long sieve_primes1   = RATPOINTS_DEFAULT_SP1;
    long sieve_primes2   = RATPOINTS_DEFAULT_SP2;
    long num_primes       = RATPOINTS_DEFAULT_NUM_PRIMES;
    long max_forbidden    = RATPOINTS_DEFAULT_MAX_FORBIDDEN;
    long b_low           = 1;
    long b_high          = height;
    long sturm_iter       = RATPOINTS_DEFAULT_STURM;
    long array_size       = RATPOINTS_ARRAY_SIZE;
    int no_check          = 0;
    int no_y              = 0;
    int no_reverse        = 0;
    int no_jacobi         = 0;
    int no_output         = 0;

    unsigned int flags = 0;

    data *info = malloc(sizeof(data));

    /* initialize multi-precision integer variables */
    for(n = 0; n <= degree; n++) { mpz_init(c[n]); }

    (...)

    { /* set up polynomial */
        long k;
        for(k = 0; k < 7; k++) { mpz_set_si(c[k], ...); }

        args.cof          = &c[0];
        args.degree       = 6;
        args.height       = height;
        args.domain       = &domain[0];
        args.num_inter    = 0;
    }
}

```

```

args.b_low      = b_low;
args.b_high     = b_high;
args.sp1        = sieve_primes1;
args.sp2        = sieve_primes2;
args.array_size = array_size;
args.sturm      = sturm_iter;
args.num_primes = num_primes;
args.max_forbidden = max_forbidden;
args.flags      = flags;

info->... = ...;
(...)

total = find_points(&args, process, (void *)info);
if(total == RATPOINTS_NON_SQUAREFREE)
{ ... }
if(total == RATPOINTS_BAD_ARGS)
{ ... }

(...)

}

/* clean up multi-precision integer variables */
for(n = 0; n <= degree; n++) {mpz_clear(c[n]); }

return(0);
}

```

If points are to be searched on many curves of the same degree, then it is slightly more efficient to use the sequence

```

args.degree = degree; /* this information is needed */
find_points_init(&args);

for( ... )
{ ...
  total = find_points_work(&args, process, (void *)info);
  ...
}

find_points_clear(&args);

```

This avoids the repeated allocation and freeing of memory.

For practical examples, see `main.c` (the code that wraps `find_points` for the command line program `ratpoints`) or `rptest.c` (which runs `find_points` on some test data).

7 Fine-tuning the parameters

7.1 How `sp1` and `sp2` are chosen

Unless they are given explicitly, the two most important parameters are chosen from the curve. While collecting the sieving information, the program computes, for each prime p , the proportion r_p of residues a modulo p for which $f(a)$ is a square, and sorts the primes by increasing r_p ; the expected proportion of numerators surviving the first k of them is then the product of the k smallest r_p . Multiplied by the number of bits that are set in a bit array to begin with, this is the expected number of survivors per bit array.

The first sieving stage costs one load and one AND per bit array per prime, whatever survives, while the second stage costs nothing on an empty bit array and a good deal on a non-empty one. So primes should be moved into the first stage until few enough bit arrays are left alive. Measurements over curves spanning a factor of 600 in density show that the best `sp1` is, to within a factor of about two, the one at which the expected number of survivors per 64-bit word falls below a constant — almost independently of the curve, even though the resulting `sp1` itself ranges from 8 to 30. That constant is `RATPOINTS_SURVIVORS_PER_WORD` in `ratpoints.h`; it is a property of the machine rather than of the curve. Per word rather than per bit array on purpose: tuning runs at 128, 256 and 512 bits give 0.0075, 0.0075 and 0.010 survivors per word, so the same constant serves every register width, whereas a per-bit-array figure would have to double with each doubling of the width. Then `sp2` is set to `sp1` plus `RATPOINTS_SP2_EXTRA`, capped by the number of primes that carry any information at all.

Because `sp1` is chosen this way, the second sieving stage now finds only a few per cent of the bit arrays non-empty whatever the curve, and it spends most of its time stepping over the empty ones. It does that in a tight loop of its own rather than re-entering the main loop body each time, which is worth some 7% on its own. Making that loop test several bit arrays at once, by or-ing them together, used to pay when `sp1` was fixed and the proportion of surviving bit arrays therefore varied a great deal; now that the proportion is held down to a couple of per cent it no longer does, since a group containing a survivor wastes its whole ‘or’, and that happens often enough to cost more than the tests it saves.

Taken together, and against the fixed values used up to version 2.2.3, this is about 14% faster on the random curves of `make test1` and about 36% faster on the point-rich curves of `make test1many`.

7.2 Retuning for your machine

For large computations, it may be a good idea to try to find the best (or at least, a good) combination of parameters for the given kind of data. The two constants above were tuned on my current laptop by minimising the sum of the times of `make test1` and `make test1many`. To repeat that on your machine, run

```
> make tune
```

which does the whole thing: it sweeps first the threshold and then the offset over both test sets, and writes the values it finds to a file `tuning.mk` that the `Makefile` includes, so that a subsequent `make all` builds with them. No source file is touched; deleting `tuning.mk` restores the values compiled into `ratpoints.h`. The file records the configuration it was measured for, and is ignored (with a warning) if you later build with a different register width or `PRIME_SIZE`, since the best values may differ.

It is deliberately not part of `make all`: it takes several minutes, wants an otherwise idle machine, and must not be run under `make -j`. How long depends on the register width, since that is what

decides how long the two tests take — on the machine this was developed on, about three minutes for the 256-bit build, four for 128-bit, six for the plain 64-bit one and eight for the 512-bit code emulated through narrower operations.

Four environment variables adjust it, should the defaults not suit: `ROUNDS` (how many times each setting is measured, default 3), `WARMUP` (seconds of load before anything is counted, default 20), `MARGIN` (how much better the winner must be, default 0.97) and `NOISE` (how much residual noise is tolerated, default 0.02). For instance `ROUNDS=6 make tune` on a machine that reports itself too noisy.

Timing this reliably is harder than it looks, and `tune.sh` goes to some trouble over it. The cost surface is flat — anything within a factor of two of a good threshold costs under 3% — whereas a laptop under sustained load slows by a quarter as it warms up, so a straight sweep would measure the clock rather than the settings. Each candidate is therefore timed back to back with the current settings and only the ratio is kept, the median over several rounds is used, and the current settings are themselves among the candidates, so that the ratio they measure against themselves shows how much noise is left. If that is more than 2% the run is declared inconclusive, and nothing is written unless the winner is better by more than 3% — a noisy run should leave a good default alone rather than replace it with a worse one.

Two consequences are worth expecting rather than being surprised by. A run that reports that nothing beat the current settings has *succeeded*: on a build that is already tuned that is the right answer, and it is what happens here. And a configuration whose tests take long enough for the machine's thermal behaviour to change within a single round — the plain 64-bit build on a laptop, for instance — may well report itself inconclusive however often it is run; raising `ROUNDS`, or using a machine that does not throttle, is then the remedy.

Should you prefer to do it by hand, the two constants can be set directly:

```
> ./rptest -r 0.0075 -R 5 -z
> ./rptest-many -r 0.0075 -R 5 -z
```

for a few values of each, minimising the sum of the two times; then put the values you found into `ratpoints.h`. The option `-T` makes the test print the CPU time it used, and nothing else, which is what `tune.sh` uses. The threshold `-r` matters considerably more than the offset `-R`, which at large height bounds hardly matters at all. Use both tests: they cover the two regimes that occur in practice, and a value that suits one of them can be a poor choice for the other.

For your machine and input data, other values may be better. You can replace the `testdata.h` file with your own collection of test data (and edit `rptest.c` if necessary to adapt the degree and height settings) and then time `./rptest -z` (the `-z` option suppresses the output) for various combinations of the parameter settings. `rptest` accepts most of the optional parameters of `ratpoints`, in particular the `-p`, `-N`, `-n`, `-r`, `-R`, `-F` and `-S` parameters, as well as `-T` (print the CPU time used, and nothing else), so you can easily change the parameters used on the command line. In addition, there is an option `-h H` to change the default height bound and an option `-m m` that causes the program to repeat the computations `m` times.

Once you have found a good set of parameter values for your application, you can hard-code them as defaults into `ratpoints` by changing the definitions in `ratpoints.h` (and then make `distclean all test`), or you can use them to fill the `ratpoints_args` structure for your call to `find_points`.

8 Implementation

8.1 Overview

Let $F(x, z)$ be the binary form of even degree corresponding to the polynomial on the right hand side of the curve equation. The basic idea is to let run b from 1 to the height bound H , for each b , let a run from $-H$ to H , and for each coprime pair (a, b) check if $F(a, b)$ is a square.

Of course, in this form, this would take a very long time. To speed up the process, we try to eliminate quickly as many pairs (a, b) as possible before the actual test. This can be done by ‘quadratic sieving’ modulo several primes: if $F(a, b)$ is a square, it certainly has to be a square mod p , and so we can rule out all (a, b) that do not satisfy this condition. Another ingredient is to represent (for a fixed b) the various values of a by bits and treat all the bits in a ‘bit array’ (of 32 or 64 bits, as the case may be, or 128, 256 or 512 bits when SSE/AVX instructions are used) in parallel. For this, we organize the sieving information for each prime p into p arrays of p words each, one such array for every $b \bmod p$, such that the j th bit in this array is set if and only if $F(j, b)$ is a square mod p .

For each ‘denominator’ b , we then set up an array of words whose bits represent the range $-H \leq a \leq H$ (aligned so that $a = 0$ corresponds to the 0th bit of a word); the bits are initially set. Then for each of the sieving primes p , we perform a bit-wise *and* operation between this array and the sieving information at p . In a first stage, this is done on the whole array; after this first stage, each remaining (‘surviving’) non-zero word in the array is subject to tests with more primes. If some bits are still set after this second stage of sieving, the corresponding pairs (a, b) (if coprime) are then checked exactly.

In the following subsections, we discuss a number of improvements that were made.

8.2 Sorting the primes

This idea is due to **Colin Stahlke**. We do not just take the first so many primes in increasing order for the sieving, but we first compute the number of points the curve has mod p for a number of primes p and then sort the primes according to the fraction of x -coordinates that give points. We then take those primes for the sieving that have the smallest fraction of ‘surviving’ x -coordinates. In this way, we need fewer sieving primes to achieve a comparable reduction of point tests.

8.3 Using connected components

We note that we can only have points when $F(a, b)$ is non-negative. If we can determine intervals on which $f(x) = F(x, 1)$ is negative, then we do not have to look for points in these intervals. The necessary computations can be performed exactly, by computing a Sturm sequence for f and counting the number of sign changes at various points, see [Coh, Thm. 4.1.10]. This can in particular tell us whether F is negative definite, in which case we have already proved that there are no rational points in the curve. If there are real zeros, we use a subdivision method in order to find a collection of intervals containing the projections of the connected components of the curve over $\mathbb{R}$.

8.4 Using 2-adic information

John Cremona suggested that in some cases, one can determine beforehand that all points will have odd ‘numerators’ a , and so we can pack the bits more tightly by only representing odd numbers. This approach can be extended. We first find all solutions mod 16 (higher powers of 2 would be possible, but not very likely to give a significant improvement). Then for every

residue class mod 16 of the denominator b , we can find the residue classes mod 16 of potential numerators a . If there are none, then we can eliminate b as a denominator altogether. If all potential a 's are even or odd (this will always be the case for even denominators), we can restrict the sieving to such numerators.

8.5 Elimination of denominators

Depending on the equation, certain denominators can be excluded. We have seen an instance of this in the previous subsection, but we can also work with odd primes.

8.5.1 Odd degree and $\pm$ monic

If the polynomial has odd degree and leading coefficient ± 1 , then the denominator has to be a square. This reduces the time complexity tremendously.

8.5.2 Odd degree general

In general, when f has odd degree, the denominator has to be 'almost' a square: it must be a square times a (squarefree) divisor of the leading coefficient, and there are further restrictions on the parity of the valuation at p , for p dividing the leading coefficient, when this valuation is sufficiently large. This gives the same type of time complexity as in the monic case, but with a larger constant.

8.5.3 Even degree with non-square leading coefficient

Here we can exclude denominators divisible by a prime p such that the leading coefficient is a non-square mod p . We can also in some cases rule out denominators divisible by a certain power of p when p is a prime that divides the leading coefficient (if the leading coefficient is not a p -adic square). While computing the sieving information, we make a list of such primes and prime powers, which we then use later to eliminate denominators. In addition, we use the necessary condition that the Jacobi symbol $(\frac{l}{b'})$ must be $+1$, where l is the leading coefficient and b' is the part of b coprime to $2l$.

8.6 Reversing the polynomial

The exclusion of denominators described above is more or less effective, depending on the situation; it is the better the earlier the case was described. Since the height of the points does not change under the transformation $(x, y) \rightarrow (1/x, y/x^{\lceil n/2 \rceil})$, we can as well search on the reversed polynomial $F(z, x)$. This will result in a speed-up if the reversed polynomial belongs to a 'better' class than the original.

8.7 Some general remarks

Modern processors are very fast when doing basic things like moving data around between registers or simple arithmetic operations (addition, subtraction, shift, $\dots$, comparison). Even memory access can be fast when there is no cache miss. On the other hand, integer division is a slow process. It turned out that quite some improvement of the performance was possible by removing as many instances of integer division operations as reasonably possible.

For example, we use gcd and Jacobi symbol routines that rely (almost) entirely on differences and shifts. We compute the residue classes of b modulo the various sieving primes by addition of the difference from the last b and then correcting by subtracting p a number of times if necessary, and

we implement most of the testing of ‘forbidden divisors’ of b using bit arrays similar to those used for sieving the numerators.

8.8 Change log

Version 2.0 was released January 9, 2008.

Version 2.0.1 was released July 7, 2008. It fixes a bug that prevented the ‘-1’ option to work properly.

Version 2.1 was released March 9, 2009. It makes use of the SSE instructions, so that the sieve can work on 128 bits in parallel (instead of on 64 or 32). On my laptop (with an Intel Core2 processor), make test runs about 25% faster than before.

Version 2.1.1 was released April 14, 2009. It fixes a bug that in some cases prevented an early abort when `*quit` was set by the callback function. Thanks to **Robert Miller** for the bug report and the fix. In addition, it is now checked that `__WORDSIZE == 64` before SSE instructions are used (in `rp-private.h`), since the program then assumes that a bit array consists of two unsigned longs. In this context, `__SSE2__` has been replaced by a new macro `USE_SSE`. A further fix eliminates unnecessary copying of sieving information when SSE instructions are not used (introduced in version 2.1). This was almost always harmless, but could have resulted in memory corruption in the extreme case that all the information had to be computed for all the primes.

Version 2.1.2 was released May 27, 2009. It fixes some memory leaks. Thanks to **Robert Miller**.

Version 2.1.3 was released September 21, 2009. The library function `find_points` should now work without any bound on the degree of the polynomial. The degree bound for the `ratpoints` executable is now set to 100 by default (specified by `RATPOINTS_MAX_DEGREE`, used to be 10).

A bug in `rpctest.c` (pointed out to me by **Giovanni Mascellani** and **Randall Rathbun**) that could lead to a segmentation fault was fixed on March 10, 2011.

Version 2.2 was released January 9, 2022. The main change is that `ratpoints` can now also work with 256-bit (and, in principle, 512-bit) registers when the CPU has the relevant capabilities. Thanks to **Bill Allombert** for doing a first implementation of this. In addition, the code has been cleaned up to some extent (e.g., the variants according to register sizes are now completely dealt with through macros defined in `rp-private.h`), and some more comments have been added. There are also some further optimizations; for example, the first sieving phase now uses many of the SSE/AVX registers in parallel, and on some architectures, the code uses a faster implementation for the test whether a bit array is zero. On my current laptop, the new code (using 256-bit words) is about twice as fast as the old one (2.1.3 using 128-bit words).

Version 2.2.1 was released January 18, 2022. This fixes a small bug that was introduced in the previous version, which could lead to the program missing points at the upper end of the search interval for the numerators. Thanks to **Bill Allombert**.

Version 2.2.2 was released May 15, 2023. This fixes a small bug that led to the polynomials being reversed incorrectly when the degree is odd. Thanks to **Nicolas Mascot** and **Bill Allombert**.

Version 2.2.3 was released September 6, 2026. It fixes the code path using 512-bit registers, which had never been exercised. Two problems were found. The first one would have made the program crash: the array `sieves0` of precomputed sieving information is declared as an array of unsigned long, but it is read through pointers to `ratpoints.bit_array`, and gcc aligns such a global array to at most 32 bytes, whereas a 512-bit bit-array needs 64. The aligned 512-bit load that gcc generates in the second sieving phase would then fault; whether it did so depended on where the linker happened to place the array. The array is now given the alignment of `ratpoints.bit_array` explicitly, in every variant. The second problem was that the `TEST` macro, which is used in the

innermost loop of the second phase, still used the generic fall-back version; gcc compiles this to a spill of the whole 64-byte vector to the stack followed by reading it back word by word, which defeats store-to-load forwarding. It now uses `vptestmq`, in analogy with the 256-bit version.

Since I have no CPU with AVX512F available, this was tested by compiling with `-DUSE_AVX512` but without `-mavx512f`, so that gcc expresses the 64-byte vectors through 256-bit operations. This exercises the whole code path (in particular `RBA_PACK == 8` and the two mask macros); `make test1` and `make test2` both reproduce the reference output exactly, as do 400 randomly generated comparisons against the 256-bit version. What this cannot check is the genuine 512-bit instructions. In the meantime, **Drew Sutherland** has run the two variants against each other on an AMD Ryzen 9 9950X3D (Zen 5 architecture), for a curve with many rational points and a height bound of 10^6 ; the 512-bit version was faster by 13 to 14%.

References

- [Coh] H. Cohen, *A course in computational algebraic number theory*, Springer GTM 138, second corr. printing, 1995.
- [gmp] The GNU Multiple Precision Arithmetic Library, <http://gmplib.org/>
- [rat] The `ratpoints-2.2.3` package. Available at
<http://www.mathe2.uni-bayreuth.de/stoll/programs/ratpoints-2.2.3.tar.gz>

MATHEMATISCHES INSTITUT, UNIVERSITÄT BAYREUTH, 95440 BAYREUTH, GERMANY.

Email address: `Michael.Stoll@uni-bayreuth.de`